

FULL STACK DEVELOPMENT

FASE 4 | AULA 02 -

SINGLE SPA - ROOT CONFIG

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Prepare-se para entrar no fascinante universo do Single SPA, uma das ferramentas mais poderosas para trabalhar com microfrontends. Nesta aula, vamos explorar em detalhes como essa tecnologia pode transformar a forma como você desenvolve e gerencia aplicações frontend, especialmente quando falamos de arquiteturas compostas por vários microfrontends.



HANDS ON

Esta aula será dividida em três vídeos, cada um focado em diferentes aspectos da configuração e uso do Single SPA para criar uma arquitetura de microfrontends. Vamos trabalhar juntos para configurar e entender cada parte dessa ferramenta poderosa.

No primeiro vídeo, vamos começar com a introdução ao Single SPA. Aqui, você aprenderá o que exatamente é o Single SPA, porque ele é tão importante na arquitetura de microfrontends, e como ele se compara com outras abordagens. Vamos também fazer a configuração inicial, instalando a CLI do Single SPA e criando um projeto básico. Este é o seu ponto de partida para começar a integrar diferentes frameworks e bibliotecas em uma única aplicação coesa.

No segundo vídeo, mergulharemos na estrutura e na arquitetura do Root Config. O Root Config é essencial para o funcionamento do Single SPA, pois é o ponto central que orquestra todas as suas aplicações independentes. Teguiaremos na criação e configuração do arquivo root-config.js, mostrando como registrar aplicativos e preparar o terreno para o crescimento modular da sua aplicação. Este vídeo é crucial para entender como organizar e gerenciar sua arquitetura de microfrontends.

Finalmente, no terceiro vídeo, vamos abordar o registro e carregamento de aplicações no Root Config. Vamos ver, passo a passo, como registrar múltiplas aplicações, garantir que elas sejam carregadas de forma independente e ainda assim funcionem juntas de maneira integrada. Também discutiremos estratégias para compartilhar dependências entre os microfrontends, minimizando a duplicação de código e melhorando a eficiência do sistema. Este último vídeo vai consolidar todo o aprendizado e te preparar para aplicar o Single SPA em projetos reais.

SAIBA MAIS

Grandes poderes, grandes responsabilidades

Vamos falar sobre um dos aspectos mais fascinantes do Single SPA: sua resiliência e adaptabilidade. Imagine poder trabalhar com diferentes frameworks e bibliotecas dentro de uma única aplicação e poder aproveitar o melhor de cada tecnologia sem estar preso a uma única escolha. É como montar um time de super-heróis, onde cada um traz suas habilidades e juntos formam algo muito maior. Isso é o que o Single SPA nos permite fazer: combinar Angular, React, Vue (ou qualquer outra tecnologia que você prefira), em uma aplicação coesa e funcional.

O Single SPA brilha justamente pela capacidade de integrar múltiplos frameworks. Ele não impõe restrições severas sobre a tecnologia que você deve usar, o que significa que você pode aproveitar as melhores características de cada uma. Por exemplo, talvez você tenha uma parte da aplicação que funciona muito bem com React, mas uma outra equipe prefere trabalhar com o Angular. Com o Single SPA, ambos os mundos podem coexistir, e a comunicação entre eles é orquestrada de maneira suave.

No entanto, como o tio Ben sabiamente disse ao Peter Parker, “com grandes poderes, vêm grandes responsabilidades”, essa liberdade de escolha também traz desafios que não podem ser ignorados. Quando você mistura diferentes frameworks e bibliotecas, o ambiente deixa de ser homogêneo. Isso pode complicar a manutenção, pois cada parte da aplicação pode ter suas próprias dependências, formas de gerenciamento de estado, e até mesmo estilos de código. Se não houver uma coordenação cuidadosa, o código pode rapidamente se tornar fragmentado e difícil de gerenciar.

Além disso, as atualizações podem se tornar mais complexas. Manter diferentes frameworks atualizados e compatíveis entre si requer atenção e testes rigorosos. O risco de um framework quebrar a compatibilidade com outro é real, e isso pode levar a um efeito dominó que impacta toda a aplicação. Sem uma estratégia clara de gestão e governança, o que começou como uma aplicação modular e flexível pode se transformar em um quebra-cabeça difícil de resolver.

Outro ponto crucial é o desempenho. Diferentes frameworks têm diferentes necessidades e comportamentos de performance. Quando eles são combinados, é preciso garantir que o tempo de carregamento, a responsividade, e a experiência do usuário final não sejam prejudicados. O Single SPA é poderoso o suficiente para gerenciar essas questões, mas a configuração e otimização corretas são essenciais.

Single SPA e aplicações estilizadas

Quando estamos trabalhando com Single SPA e microfrontends, uma das áreas que merece muita atenção é a dos estilos CSS. Em um ambiente onde múltiplos microfrontends, potencialmente desenvolvidos por equipes diferentes e usando frameworks distintos se encontram, manter a consistência visual e evitar conflitos de estilos se torna uma tarefa desafiadora, mas essencial. A seguir, vamos falar sobre algumas estratégias e boas práticas para lidar com CSS no contexto de Single SPA, e a importância de ter um design system bem estruturado.

A primeira coisa é entender que o CSS, por natureza, é global. Isso significa que estilos aplicados em um microfrontend podem facilmente vazar para outros, causando conflitos inesperados. Por exemplo, imagine que você tem um componente h2 estilizado em azul em um microfrontend e um componente h2 estilizado em preto em outro. Sem um controle cuidadoso, esses estilos podem colidir, resultando em uma experiência de usuário inconsistente. Para evitar isso, é fundamental adotar uma estratégia de encapsulamento de estilos.

Uma das maneiras mais eficazes de encapsular estilos é utilizando CSS Modules ou Styled Components. CSS Modules, por exemplo, permitem que você escreva estilos que são escopados automaticamente ao componente onde são utilizados, prevenindo o vazamento de estilos para outras partes da aplicação. Isso é especialmente útil em um ambiente de microfrontends, onde cada equipe pode estar utilizando diferentes bibliotecas de estilo. Com CSS Modules, o estilo de um microfrontend fica contido dentro dele, evitando surpresas desagradáveis.

Outra estratégia é o uso de Web Components, que também oferecem encapsulamento nativo de estilos. Web Components permitem que você defina um “shadow DOM”, onde os estilos aplicados não afetam o restante da página, criando uma barreira natural contra conflitos de CSS. Isso pode ser uma excelente escolha se

you need to guarantee that the styles of a microfrontend do not interfere in any way with the others.

Now, talking about CSS in microfrontends without mentioning the importance of a design system would be neglecting a key piece of the puzzle. A design system is like a living style guide that unites designers and developers around a common visual and functional vision. It defines not only colors, fonts, and spacings, but also how components should be constructed and used. When you have a well-structured design system, microfrontends can benefit from a consistent set of components and styles, which helps maintain visual cohesion, even when different teams are contributing to the application.

Besides that, a design system can include design tokens — values that can be reused like colors, font sizes, and spacings — that guarantee that all microfrontends share the same visual language. This not only simplifies maintenance, but also allows for quick adaptation when design changes are necessary, since changing a single token can propagate the changes throughout the entire application.

In the end, it's important to remember that, when working with microfrontends, communication between teams and clear documentation of styles and components is fundamental. Even with all the encapsulation strategies and a robust design system, without good coordination, the risk of inconsistency still exists.

Therefore, when using Single SPA, it's essential to have a careful and strategic approach to CSS. Encapsulating styles, implementing an effective design system, and ensuring fluid communication between teams are fundamental steps to create an application that not only works well, but also provides a cohesive and pleasant user experience.

Single SPA e boas práticas

Single SPA is a powerful tool for creating microfrontends, but at the heart of all this is the root config. If Single SPA were an orchestra, the root config would be the conductor, responsible for ensuring that each microfrontend plays its part in the symphony of a harmonious and synchronized form. Let's explore why the

root config é tão crucial, quais são as suas principais funções e como garantir que ele esteja sempre atualizado e bem implementado.

O root config, basicamente, é o ponto de entrada da aplicação Single SPA. Ele é o primeiro arquivo a ser carregado e tem a missão de orquestrar todo o processo de inicialização dos microfrontends, registrando e carregando cada um conforme necessário. Uma de suas principais funções é garantir que os microfrontends sejam carregados apenas quando precisam ser exibidos, o que ajuda a otimizar o desempenho da aplicação. Além disso, ele define as rotas que cada microfrontend vai ocupar, permitindo que diferentes partes da aplicação sejam renderizadas com base na URL.

Um dos pontos mais importantes a se considerar ao trabalhar com o root config é a flexibilidade que ele oferece. Com ele, você pode misturar e combinar diferentes frameworks e bibliotecas na mesma aplicação, decidindo quais microfrontends serão carregados com base na lógica de negócios ou no comportamento do usuário. Isso significa que você pode, por exemplo, carregar um microfrontend em Angular para uma parte da aplicação e um em React para outra, sem que um interfira no outro.

Além disso, o root config permite a compartimentação de funcionalidades, o que significa que você pode dividir a aplicação em partes menores e mais gerenciáveis. Isso é especialmente útil em ambientes de desenvolvimento grandes, onde diferentes equipes podem estar trabalhando em diferentes partes da aplicação ao mesmo tempo.

Mas com toda essa flexibilidade e poder vem a necessidade de uma boa implementação. Como o root config é o núcleo da aplicação Single SPA, ele precisa ser mantido limpo e bem-organizado. Um root config mal implementado pode levar a problemas de desempenho, dificuldades de manutenção e até mesmo bugs na aplicação. Além disso, é importante garantir que você esteja sempre atualizado com a última versão do Single SPA. O ecossistema de microfrontends está em constante evolução, e novas funcionalidades e melhorias de performance são introduzidas regularmente. Manter o root config atualizado garante que você possa aproveitar essas melhorias e manter sua aplicação moderna e eficiente.

Outro aspecto crucial é a segurança. O root config é responsável por orquestrar todas as partes da aplicação, então qualquer vulnerabilidade aqui pode ter

implicações em toda a arquitetura. Certifique-se com frequência de seguir as melhores práticas de segurança e revisão de código para proteger a aplicação.

Para garantir que sua implementação do root config esteja sempre em sua melhor forma, são recomendadas revisões periódicas e testes abrangentes. O root config deve ser visto como uma peça central da aplicação, e qualquer alteração deve ser cuidadosamente planejada e implementada. Isso também significa que a documentação e o treinamento das equipes devem estar sempre atualizados, garantindo que todos estejam na mesma página e entendam como o root config funciona e como ele deve ser mantido.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, começamos com uma introdução ao Single SPA, onde discutimos o que essa ferramenta oferece e porque ela é tão crucial para quem trabalha com microfrontends. Você aprendeu como o Single SPA permite que diferentes frameworks e bibliotecas coexistam em uma única aplicação, oferecendo uma flexibilidade que poucas outras soluções proporcionam. Exploramos também a configuração inicial do Single SPA, dando o pontapé inicial na criação de um projeto com a CLI da ferramenta.

Avançamos para a estrutura e arquitetura do Root Config, que é o verdadeiro coração do Single SPA. Discutimos o papel vital que o Root Config desempenha, garantindo que todas as partes da aplicação funcionem em harmonia. Em nossa videoaula você viu como criar e configurar o arquivo root-config., aprendendo a registrar aplicativos de maneira que eles sejam carregados de forma modular e eficiente, dependendo da navegação do usuário.

Finalmente, colocamos em prática esses aprendizados com o registro e carregamento de aplicações. Exploramos passo a passo como registrar múltiplas aplicações dentro do Root Config e como carregá-las de forma independente. Além disso, abordamos estratégias para compartilhar dependências entre diferentes microfrontends, o que é fundamental para evitar a duplicação de código e garantir uma aplicação mais eficiente e fácil de manter.

REFERÊNCIAS

SINGLE-SPA Documentation. **Create Single SPA**. Disponível em: <https://single-spa.js.org/docs/create-single-spa> . Acesso em: 14 ago. 2024.

SINGLE-SPA Documentation. **Recommended Setup**. Disponível em: <https://single-spa.js.org/docs/recommended-setup> . Acesso em: 14 ago. 2024.

SINGLE-SPA Documentation. **Ecosystem CSS**. Disponível em: <https://single-spa.js.org/docs/ecosystem-css> . Acesso em: 14 ago. 2024.

PALAVRAS-CHAVE

Modularidade. Microfrontends. Arquitetura Frontend.

EMSE

POSTECH